

Lab0: GitLab

TODO. 1

1.1. 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

我之前没有正式的多人协同开发经历。如果不使用版本控制工具，多人协作一般是互相传输代码文件，很容易出现代码覆盖、版本混乱，无法追溯每一处修改。

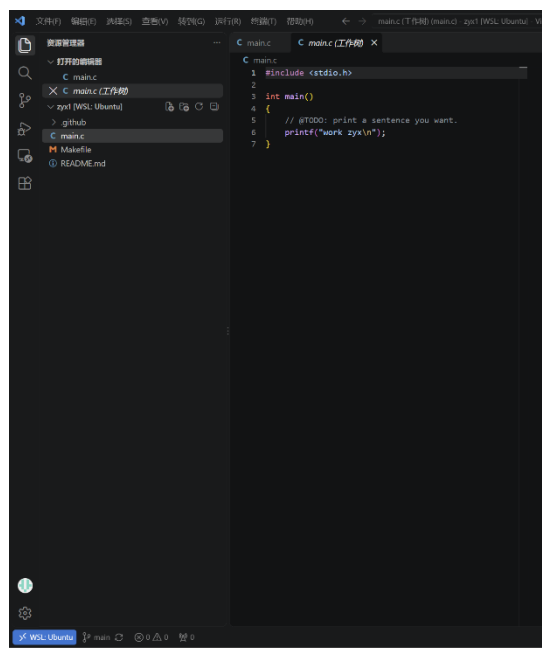
1.2. 思考一下，Git 为什么要设计“暂存-提交”两个步骤？

设置暂存区，可以让开发者选择性挑选需要提交的改动，保证每次提交是一个完整独立的变更，让版本历史更加整洁，方便后续查看、回滚代码。

1.3. `git branch` 和 `git branch -a` 的区别是什么？

前者只列出本地仓库的分支，不显示远程分支；后者：`-a` 代表 `all`，会同时列出本地分支以及远程仓库上的所有分支。

TODO. 2



TODO. 3

1. Commit Message 规范

Commit Message 是每次代码提交时填写的说明信息。规范的提交信息要求清晰描述本次代码改动的目的、改动内容，不能只写模糊简单的文字。

遵循规范可以快速看懂每一次提交做了什么，方便后续查看版本历史、筛选提交记录、定位 bug，也便于团队成员之间互相理解代码变更。

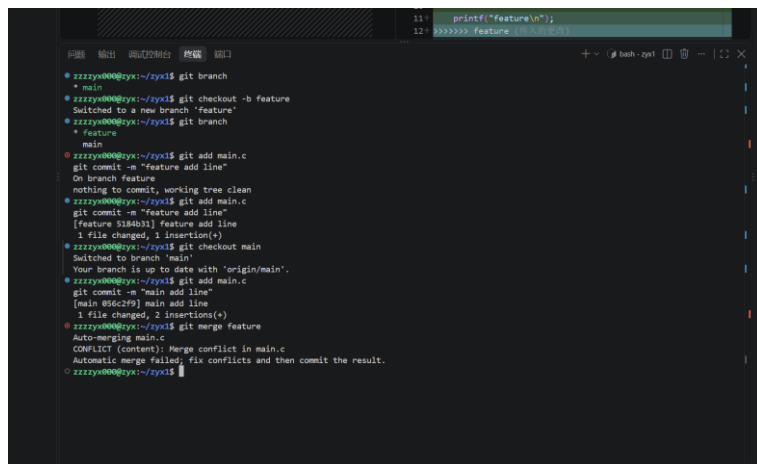
2. Git Flow 分支控制

GitFlow 是一套标准化的 Git 分支管理工作规范，用来规范团队多人协作开发。它把分支分为永久分支和临时分支，master 保存稳定上线代码；develop 作为开发主分支；feature 分支用来开发新功能；release 分支用于版本发布前测试；hotfix 分支用来紧急修复线上生产 bug。减少多人开发时代码冲突，让项目迭代有序可控。

Git 是主流分布式版本管理工具。方便多人并行开发，减少代码冲突。规范的提交说明便于追溯改动、排查问题，是团队协作开发必不可少的基础工具。

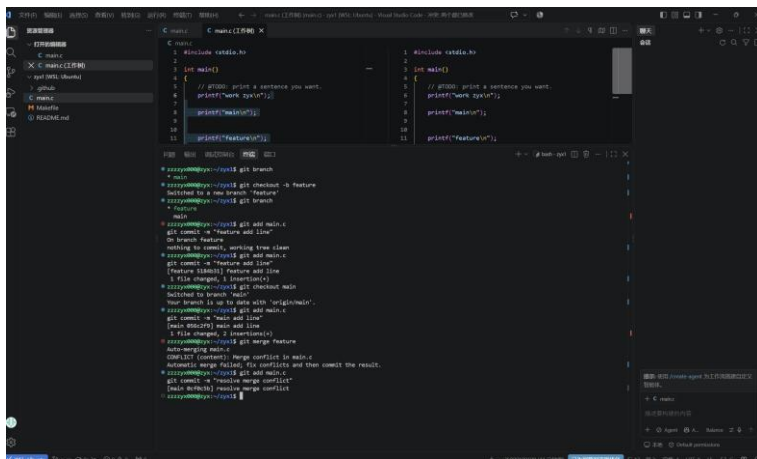
TODO. 4

我在同样位置输入合并后冲突，
修改冲突方法为在文件夹里删去了错误的式子。



```
11: printf("feature\n");
12: >>>>> feature

zzzyy@zyx:~/zyx$ git branch
* main
zzzyy@zyx:~/zyx$ git checkout -b feature
Switched to a new branch 'feature'
zzzyy@zyx:~/zyx$ git branch
* feature
  main
zzzyy@zyx:~/zyx$ git add main.c
git commit -m "feature add line"
On branch feature
nothing to commit, working tree clean
zzzyy@zyx:~/zyx$ git add main.c
git commit -m "feature add line"
[feature 5184031] feature add line
1 file changed, 1 insertion(+)
zzzyy@zyx:~/zyx$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
zzzyy@zyx:~/zyx$ git add main.c
git commit -m "main add line"
[main 8f6c219] main add line
1 file changed, 2 insertions(+)
zzzyy@zyx:~/zyx$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
zzzyy@zyx:~/zyx$
```



```
C main
1 #include <stdio.h>
2
3 int main()
4 {
5     // printf a sentence you want.
6     printf("main\n");
7
8     printf("feature\n");
9
10    printf("feature\n");
11
12    return 0;
13 }
```

```
C feature
1 #include <stdio.h>
2
3 int main()
4 {
5     // printf a sentence you want.
6     printf("main\n");
7
8     printf("feature\n");
9
10    printf("feature\n");
11
12    return 0;
13 }
```

```
zzzyy@zyx:~/zyx$ git branch
* feature
  main
zzzyy@zyx:~/zyx$ git checkout -b feature
Switched to a new branch 'feature'
zzzyy@zyx:~/zyx$ git branch
* feature
  main
zzzyy@zyx:~/zyx$ git add main.c
git commit -m "feature add line"
On branch feature
nothing to commit, working tree clean
zzzyy@zyx:~/zyx$ git add main.c
git commit -m "feature add line"
[feature 5184031] feature add line
1 file changed, 1 insertion(+)
zzzyy@zyx:~/zyx$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
zzzyy@zyx:~/zyx$ git add main.c
git commit -m "main add line"
[main 8f6c219] main add line
1 file changed, 2 insertions(+)
zzzyy@zyx:~/zyx$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
zzzyy@zyx:~/zyx$ git add main.c
git commit -m "feature merge conflict"
[main 8f6c219] feature merge conflict
1 file changed, 2 insertions(+)
zzzyy@zyx:~/zyx$
```